07 — Infrastructure & DevOps

07 — Infrastructure & DevOps

Revision: 1 **Last modified:** 2026-07-05T14:20:00Z **Status:** Draft — consolidated from `05-repo-layout-tooling-and-helix-ecosystem.md` and `v06-deploy/*`.

---

1. Position and scope

This section owns the **physical engineering substrate** of HelixVPN: the monorepo layout, the reusable-component extraction plan, schema-first code generation, the `helixvpnctl` operator CLI, the three deployment substrates (Podman quadlets, Docker Compose, Kubernetes), and the wiring of every incorporated `submodules/` member into a concrete role.

It does not own the logical architecture — data plane, control plane, client core, security — only the files, repos, build tools, and deploy units that materialize them.

---

2. Repo layout strategy: monorepo now, extract later

The MVP builds in one working monorepo `helixvpn/` whose internal workspace boundaries are already the future repo boundaries. Once the public surface of each pillar stabilizes, the pillar is extracted to a standalone `vasic-digital` repo and re-consumed as a flat submodule.

Pillar	Monorepo dir	Future repo	Extract trigger
Schema contract	`helix-proto/`	`helix_proto`	First frozen `WatchNetworkMap` version
	`helix-core/`	`helix_core`	

Pillar	Monorepo dir	Future repo	Extract trigger
Rust client/ connector core			iOS memory gate passes + FFI surface freezes
Rust gateway edge	`helix-edge/`	`helix_edge`	Right after `helix_core`
Flutter UI	`helix-ui/`	`helix_ui`	`runHelixApp` flavors + status-stream contract stable
Go control plane	`helix-go/`	`helix_control`	Policy compiler + coordinator stable (last code repo)
Deployment/ tooling	`deploy/`	`helix_deploy`	Image names + env contract freeze
Design system	already submodule	`helix_design`	Decoupled day one (Volume 10)

Extraction is operator-gated via `gh repo create` / `glab repo create`; history is preserved with `git filter-repo` and the new repo is immediately added as a flat submodule. Release tags across every repo carry the `helix_vpn-` prefix (§11.4.151).

---

3. Decoupling and dependency-from-root

Reusable repos are **project-not-aware**: no HelixVPN hostnames, asset names, or tenant assumptions are hardcoded. Project specifics enter only through config injection (env var / config struct / constructor param). The dependency graph is flat — every own-org submodule is reachable from the parent root, never nested inside another submodule.

Invariant	Enforcement
Project-not-aware	`CM-OWNED-SUBMODULE-DECOUPLING` gate greps submodule diffs for parent context
Flat dependency layout	`CM-OWNED-SUBMODULE-LAYOUT` gate forbids nested own-org `.gitmodules` chains
Equal engineering	Each owned submodule gets the same test/ doc/anti-bluff attention
Catalogue-first reuse	`Catalogue-Check: reuse

---

4. Schema-first codegen (zero-drift contract)

All Go/Dart/Rust/TS clients are generated from canonical schemas; hand-written clients are forbidden.

Plane	Canonical source	Generators	Outputs
Agent plane	`proto/helix/coordinator/v1/*.proto`	`buf` (Connect: gRPC + gRPC-Web + Connect)	Go server stubs, Dart/Rust clients
App plane	`openapi/helix-rest.v1.yaml`	`oapi-codegen`, `openapi-generator`, `openapi-typescript`	Go Gin stubs, Dart/TS clients

The local build contract:

- `make gen` — regenerate all clients from schema.
- `make verify-gen` — regenerate, then `git diff --exit-code`; any drift blocks the commit.
- `buf lint` + `buf breaking --against main` + `oasdiff breaking` run in the local pre-build sweep.
- Generated code is gitignored; the schema is the single source of truth.

Protovalidate constraints travel with the `.proto` so a 31-byte `wg_pubkey` is rejected in every generated language.

---

5. `helixvpnctl` — the operator CLI

`helixvpnctl` is the single Go binary that replaces the original pile of bash install scripts. It has two front doors:

- **Homelab front door** — `helixvpnctl init --domain gw.example.com` bootstraps a single-node gateway (tenant, admin, CA, keys, creds, deploy units), then prints `systemctl --user start helixvpn-pod`.
- **GitOps front door** — `helixvpnctl policy set ./policy.jsonc` dry-run-compiles, persists a new policy version, and optionally activates it atomically.

Subcommand	Purpose
`init`	Bootstrap gateway; default substrate is quadlet
`keys rotate/show`	Gateway WireGuard key operations
`enroll-token`	

Subcommand	Purpose
	Mint single-use, TTL'd device enrollment token
`policy get/set/compile`	GitOps policy front door; `set` is fail-closed
`device list/revoke`	Device lifecycle; revoke < 1 s
`connector list/advertise`	Declarative advertised CIDRs
`status`	Gateway/edge health + transport ladder
`deploy quadlet/compose/kube`	Render substrate units

Credentials are handled per §11.4.10: the operator token is never logged, token files are mode `0600`, and loose-perm files are refused.

---

6. Deployment substrates

All three substrates run the **same** OCI images (`helixd`, `helix-edge`, `postgres:16`, `redis:7`) with the **same** env contract. They are generated from one in-code spec via the `containers` submodule so a stack-shape change lands in all three, not three hand-maintained files.

6.1 Podman quadlets — canonical, rootless

Podman rootless quadlets are the §11.4.161-mandated canonical substrate. A single pod groups edge + control + Postgres + Redis.

Concern	Rule
Runtime	Rootless Podman only — no `sudo`, no rootful Docker for container ops
Unit location	`~/.config/containers/systemd/` (rootless search path)
Edge capability	`DropCapability=ALL`; add only `NET_ADMIN` + `NET_RAW` + `/dev/net/tun`
Control/DB/Redis	`DropCapability=ALL`; no extra caps
Secrets	Podman `Secret=` injected as env target; never in unit text
Ingress	Pod publishes `:443/udp` (MASQUE) + `:51820/udp` (plain WG)

Concern	Rule
Host prerequisites	`net.ipv4.ip_unprivileged_port_start=443`; WireGuard kernel module loaded

6.2 Docker Compose — documented fallback

For operators already running Docker. Same images and env contract, mapped service-by-service to a `compose.yaml`. Rootless Docker is preferred; rootful Docker is an honest §11.4.112-documented higher-privilege gap, never presented as equivalent to the canonical path.

6.3 Kubernetes — fleet / Phase-2 HA

Plain manifests + kustomize overlays for the multi-region fleet profile. Key shapes:

Component	K8s shape	Why
`helixd`	`Deployment` + HPA	Stateless coordinator; safe to scale/restart
Postgres	`StatefulSet` / Patroni / CloudNativePG	Single durable source of truth (C2)
Redis/NATS	`StatefulSet` (ephemeral)	Event + presence backing; loss is recoverable
`helix-edge`	`DaemonSet` + `hostNetwork: true`	UDP/443 must land on node IP; kernel-net caveat
Network policy	Default-deny mesh	Calico/Cilium-class CNI required

The edge capability set is the same canonical `{NET_ADMIN, NET_RAW}` + `/dev/net/tun` across all substrates.

---

7. Ecosystem submodule integration

Every incorporated `submodules/` member has a concrete, catalogue-first role:

Submodule	Role	Catalogue-Check
`containers`	**Sole** container orchestration layer: deploy render + on-demand test infra	reuse

Submodule	Role	Catalogue-Check
`helix_qa`	Anti-bluff QA orchestrator driving the 8 MVP-DoD criteria	reuse
`challenges`	Challenge engine; HelixVPN authors Challenge definitions	reuse
`docs_chain`	Mechanical `final/*.md` → HTML/PDF/DOCX + workable-items DB sync	reuse
`security`	Control-plane defensive libs: PII redaction, headers, at-rest crypto, SSRF-deny, edge privesc scan	reuse/extend
`helix_design`	OpenDesign-based design system (Volume 10)	extend
`vision_engine` + `panoptic`	UI recording/vision bridge for Flutter Challenges	reuse (test-infra only)
`doc_processor`	Feature-map extraction → Status ledger cross-check	reuse
`llm_orchestrator` / `llms_verifier`	Dev-loop only — not product runtime	dev-only
`llm_provider`	Conditional Phase-2 policy-authoring helper	Phase-2 opt-in

Submodules are consumed by reference, never copied. The `containers` submodule's `pkg/boot`/`pkg/compose`/`pkg/health` primitives are the only sanctioned path to docker/podman/k8s in the codebase.

---

8. Observability, DR, and HA

8.1 Observability

The observability stack consumes control-plane series emitted by `internal/telemetry` and edge series from the Rust edge. It is counts/gauges only — no per-packet, per-flow, or per-destination data (C3).

Concern	Rule
Scrape target	`helixd:9090/metrics` internal-only; never on public `:443`

Concern	Rule
Headline SLO	`helix_reconcile_seconds` p99 < 1 s
Security SLO	`helix_revoke_enforce_seconds` p99 < 1 s
Forbidden labels	No `tenant_id`, `device_id`, `src_ip`, `dst_ip`
Dashboards	Grafana JSON-as-code under `deploy/grafana/`

8.2 Disaster recovery

DR is simple because there is exactly one durable source of truth: Postgres.

Asset	Backed up?	Rationale
Postgres + WAL	Yes	The only durable truth (C2)
CA root / PKI secrets	Yes, separately KMS-encrypted	Catastrophic if lost or leaked
Redis/NATS	No	Ephemeral; restoring would create a connection log (C3 violation)
Coordinator graph	No	Rebuilt from Postgres on boot
Deploy manifests	No	Regenerated via `helixvpnctl deploy`

RTO target < 15 min for Postgres restore or whole-region failover; RPO is 0 for sync-committed transactions, otherwise bounded by WAL archive interval.

8.3 High availability

HA is cheap because coordinators are stateless on disk.

Failure	Data-plane effect	Control-plane effect
`helixd` replica loss	None (C1)	Streams reconnect and resume by `known_version`
Postgres primary failover	None (tunnels keep forwarding)	Writes paused seconds; `/readyz` NOT-READY
Redis/NATS blip	None	Event delivery paused; outbox drains on recovery
Regional edge loss	Clients on that gateway drop	`gateway.failover` delta re-homes clients

Phase-2 multi-region uses a single Postgres primary, stateless `helixd` replicas behind a load balancer, per-region `helix-edge` DaemonSets, and geoDNS/anycast for gateway selection.

---

9. Cross-references

- Repo layout & decoupling → `../../v06-deploy/repo-layout-and-decoupling.md`
- Codegen pipeline → `../../v06-deploy/codegen-pipeline.md`
- `helixvpnctl` CLI → `../../v06-deploy/helixvpnctl.md`
- Podman quadlets → `../../v06-deploy/podman-quadlets.md`
- Docker Compose → `../../v06-deploy/docker-compose.md`
- Kubernetes → `../../v06-deploy/kubernetes.md`
- Ecosystem integration → `../../v06-deploy/helix-ecosystem-integration.md`
- Observability → `../../v06-deploy/observability.md`
- Disaster recovery → `../../v06-deploy/disaster-recovery.md`
- HA & multi-region → `../../v06-deploy/ha-and-multiregion.md`
- Source overview → `05-repo-layout-tooling-and-helix-ecosystem.md`
- Data plane → 03 — Data Plane
- Control plane → 04 — Control Plane
- Security/PKI → 06 — Security, Privacy & PKI

Sources: `docs/research/mvp/final/05-repo-layout-tooling-and-helix-ecosystem.md`, `v06-deploy/*.md`.